

Institucionalização do reuso e barreiras culturais, técnicas e gerenciais

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- O que é institucionalizar o reuso
- Barreiras ao reuso
- Estratégias para superação
- Dinâmica: Barreiras ao Reuso na Vida Real

Conceito de institucionalização do reuso

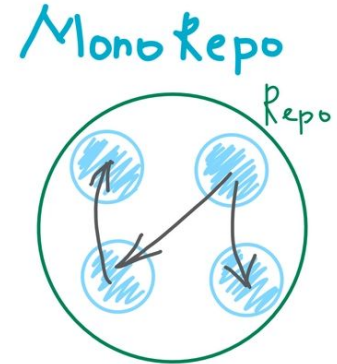
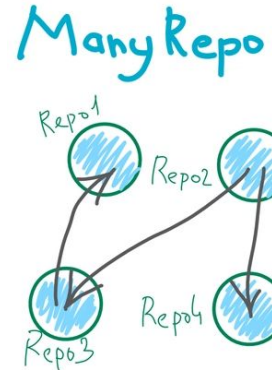
- Reuso ad hoc x reuso institucionalizado.
- Relação com maturidade organizacional:
 - Nível 0: improviso (copiar/colar).
 - Nível 1: bibliotecas internas.
 - Nível 2: processos formais de gestão do reuso.
 - Nível 3: reuso integrado à estratégia da empresa (ex.: linhas de produto).
- Não basta reusar ad hoc: precisa haver processo, cultura e suporte organizacional.
- Referências IEEE 1517 e ISO 12207 (ciclo de vida de reuso).

Conceito de institucionalização do reuso

- Segundo IEEE 1517 e literatura clássica (Mili, Krueger, Morisio):
 - **Políticas:** incentivo ao uso de componentes internos.
 - **Processos:** ciclo de vida do reuso (identificar → catalogar → adaptar → manter).
 - **Recursos:** repositórios, catálogos, ferramentas (npm, Maven, PyPI internos).
 - **Cultura:** colaboração, confiança em artefatos produzidos por outros times.

Exemplos de institucionalização

- **Google:** monorepo, revisão de código, bibliotecas internas amplamente usadas.
- Um monorepo é uma estratégia de desenvolvimento de software onde o código de múltiplos projetos é armazenado num único repositório de controle de versão, ao contrário da abordagem multi-repo onde cada projeto tem o seu próprio repositório.



Exemplos de institucionalização

- **Amazon:** APIs internas obrigatórias (política de Bezos de 2002: tudo exposto como serviço).
- **Amazon API Gateway** é um serviço que permite a criação e gestão de APIs escaláveis, servindo como "porta de entrada" para dados e funcionalidades internas da AWS.
- **SP-API** é um conjunto de APIs que permite aos parceiros de vendas gerenciar operações comerciais de forma mais eficiente.

Exemplos de institucionalização

- **Microsoft:** reuso massivo em Windows/Office (componentes compartilhados).
- **Arquitetura em nuvem reutilizável:** Serviços como OneDrive, SharePoint e Microsoft 365 interagem de forma integrada, permitindo reutilização de funcionalidades em múltiplos dispositivos e navegadores.
- **Modelos de compartilhamento e ativação:** A assinatura Microsoft 365 permite que várias pessoas reutilizem o mesmo pacote de aplicativos, enquanto a ativação em nuvem possibilita reuso eficiente de instalações em computadores partilhados.

Barreiras ao reuso

- As barreiras ao reuso de software são discutidas na literatura clássica
 - Krueger
 - Mili
 - Morisio
 - IEEE 1517
 - e também observadas em empresas na prática.
- Elas podem ser organizadas em técnicas, organizacionais/culturais e gerenciais

Barreiras Técnicas

- **Falta de padronização:** diferentes linguagens, frameworks e estilos de programação dificultam reaproveitar artefatos.
- **Documentação insuficiente:** componentes sem guias claros ou exemplos de uso reduzem a confiança e aumentam o custo de integração.
- **Baixa qualidade dos artefatos:** código pouco testado, acoplado ou sem manutenção desencoraja o reuso.
- **Problemas de compatibilidade:** versões diferentes de bibliotecas, APIs ou plataformas que não são retrocompatíveis.
- **Esforço de adaptação:** necessidade de ajustes extensivos para adequar um componente genérico a um caso específico.

Exemplos de Barreiras Técnicas

- **Falta de padronização:** em uma empresa de fintech, diferentes times usam Java, Python e .NET sem guias de estilo; um módulo de autenticação feito em .NET não é aproveitado pelos projetos em Python.
- **Documentação insuficiente:** biblioteca interna de relatórios exportados em PDF não tem manual nem exemplos. Os desenvolvedores demoram mais para entender a API do que criar uma solução nova.
- **Baixa qualidade dos artefatos:** um componente de cálculo fiscal foi feito sem testes automatizados; cada equipe prefere reimplementar as regras para evitar riscos de erro.
- **Problemas de compatibilidade:** versão antiga de um módulo de pagamentos não funciona nas novas versões do framework Spring Boot, inviabilizando o reuso em projetos modernos.

Barreiras Culturais

- **Resistência à mudança:** equipes preferem "reinventar" soluções em vez de confiar em código de outros times.
- **Síndrome do "Not Invented Here" (NIH):** desvalorização de soluções externas ou de outros grupos internos.
- **Falta de cultura de colaboração:** ausência de incentivos para compartilhar código, documentação e boas práticas.
- **Desconfiança em artefatos de terceiros:** medo de depender de componentes que podem ter bugs ou falta de suporte.

Exemplos de Barreiras Culturais

- **Resistência à mudança:** equipe de um sistema legado em COBOL prefere manter sua própria biblioteca de relatórios em vez de migrar para a solução corporativa em Java.
- **Síndrome do "Not Invented Here" (NIH):** time de desenvolvimento em uma startup rejeita usar uma API open source de geolocalização e cria uma interna, mesmo que inferior.
- **Falta de cultura de colaboração:** em uma grande empresa, não existe canal para troca de componentes entre equipes, cada time mantém seu repositório privado no Git.
- **Desconfiança em artefatos de terceiros:** engenheiros evitam bibliotecas públicas de criptografia por receio de falhas, criando implementações próprias que podem ser até menos seguras.

Barreiras Gerenciais

- **Falta de políticas institucionais:** ausência de processos formais para catalogar, aprovar e manter repositórios reutilizáveis.
- **Custos iniciais de investimento:** criar infraestrutura de reuso (catálogos, repositórios, governança) exige tempo e recursos.
- **Medição de benefícios difícil:** gestores têm dificuldade em quantificar o retorno sobre o investimento em reuso.
- **Prioridade por entregas rápidas:** pressão por prazos curtos leva ao "copia e cola" em vez de reuso planejado.

Exemplos de Barreiras Gerenciais

- **Falta de políticas institucionais:** empresa não define regras para revisar e catalogar componentes reutilizáveis; os times criam código duplicado sem controle central.
- **Custos iniciais de investimento:** um banco decide não criar um repositório interno de microserviços porque o custo de governança e manutenção seria alto, mesmo com ganho futuro.
- **Medição de benefícios difícil:** gestores não conseguem provar em números que reusar módulos de autenticação economiza tempo em projetos futuros, então o investimento não é priorizado.
- **Prioridade por entregas rápidas:** uma equipe sob pressão de prazo copia e cola um módulo de login de outro sistema em vez de adaptar a versão reutilizável corporativa, gerando duplicação.

Quadro comparativo – Barreiras ao Reuso de Software

Técnicas	Culturais	Gerenciais
Falta de padrão	Resistência à mudança	Falta de políticas
Pouca doc.	“Not Invented Here”	Custo inicial alto
Baixa qualidade	Falta de colaboração	Benefícios difíceis de medir
Incompatibilidade	Desconfiança	Pressão por prazos

Estratégias para superação

- As barreiras ao reuso de software, sejam elas técnicas, culturais ou gerenciais, podem comprometer a eficiência e a qualidade dos projetos de desenvolvimento.
- Superá-las exige estratégias bem definidas que vão além da simples criação de componentes reutilizáveis.
- É preciso alinhar práticas de padronização, promover mudanças culturais que incentivem a confiança e a valorização do reuso, e adotar políticas gerenciais que garantam tempo e recursos para refatoração e manutenção de artefatos.
- Dessa forma, as organizações podem transformar o reuso em uma prática sistemática, gerando benefícios reais de produtividade e inovação.

Técnicas

- **Padronização de interfaces**

- Adotar padrões como OpenAPI (para REST), GraphQL ou gRPC garante que serviços diferentes possam conversar entre si.
 - Exemplo real: o GitHub padronizou sua API em GraphQL, facilitando integrações de terceiros e reduzindo retrabalho.

- **Testes automatizados obrigatórios**

- Componentes só entram no catálogo de reuso se possuírem testes unitários e de integração.
 - Exemplo real: o Google exige cobertura mínima em testes antes de aprovar bibliotecas internas para uso em outros projetos.

Técnicas

- **Catálogo interno de pacotes**
 - Uso de ferramentas como Artifactory, Nexus, GitHub Packages, npm interno.
 - Permite buscar e baixar bibliotecas internas como se fossem pacotes externos.
 - Exemplo real: a Microsoft mantém um “One Engineering System” centralizado, que distribui pacotes reutilizáveis para todos os times do Windows e Office.
- **Versionamento e compatibilidade**
 - Garantir que bibliotecas tenham versões estáveis e backward-compatible.
 - Exemplo real: o Spring Framework mantém versões LTS (long term support) para permitir reuso seguro em grandes corporações.

Culturais

- **Reconhecimento e incentivo**
 - Criar métricas e recompensas para equipes que produzem código reutilizável (promoções, bônus, menções).
 - Exemplo real: na SAP, desenvolvedores que criam componentes que viram “corporate standard” recebem visibilidade interna.
- **Comunidades internas de prática (guildas, chapters)**
 - Estruturar fóruns de troca de conhecimento, como guildas de front-end ou chapters de DevOps.
 - Exemplo real: o Spotify organizou guildas para alinhar práticas de UI e APIs, reduzindo duplicação de soluções.

Culturais

- **Divulgar casos de sucesso internos**
 - Mostrar exemplos onde o reuso trouxe economia real de tempo e esforço, gerando confiança.
 - Exemplo real: a Uber promove “Tech Talks” onde engenheiros apresentam como reutilizaram bibliotecas internas para lançar features mais rápido.

- **Combater o “Not Invented Here”**
 - Criar cultura de confiança em soluções externas bem estabelecidas (open source, bibliotecas corporativas).
 - Exemplo real: a Netflix criou diversas ferramentas open source (Hystrix, Zuul, Eureka) que depois foram reutilizadas por milhares de empresas, inclusive internamente.

Gerenciais

- **Políticas claras de reuso**
 - Definir que todo novo sistema deve avaliar componentes existentes antes de “reinventar a roda”.
 - Exemplo real: no Banco Itaú, há governança de APIs: antes de criar uma nova, o time deve verificar se já existe API similar no catálogo corporativo.
- **Tempo e orçamento para refatoração**
 - Dedicar “sprints de refatoração” para extrair componentes genéricos e reaproveitáveis.
 - Exemplo real: a Amazon investe tempo em “two-pizza teams” para transformar soluções locais em serviços utilizáveis globalmente.

Gerenciais

- **Medição de resultados**

- Indicadores como:

- % de código reutilizado em projetos.
 - Redução no tempo de entrega de novas funcionalidades.
 - Diminuição de bugs por reuso de componentes testados.
 - Exemplo real: startups de fintech reportam que reutilizando APIs open source + SaaS (ex.: Stripe, Twilio) reduziram o tempo de entrega de MVPs de meses para semanas.

- **Infraestrutura de suporte ao reuso**

- Investir em CI/CD, governança e documentação.

- Exemplo real: a Google Cloud mantém guidelines para todos os times publicarem bibliotecas internas com pipelines padronizados.

Exemplos

- **Spotify:** guildas e chapters para alinhar práticas → supera barreiras culturais.
- **Netflix:** ferramentas OSS (Hystrix, Zuul, Eureka) → reutilizadas globalmente → supera barreiras técnicas e culturais.
- **Microsoft:** integração Windows/Office e catálogo interno → supera barreiras técnicas com massivo reuso.
- **Startups:** combinam open source + APIs de terceiros (ex.: Firebase, Stripe, Twilio) → superam barreiras gerenciais de tempo e custo.

Quadro comparativo

Técnicas	Culturais	Gerenciais
Definir padrões e boas práticas	Incentivar colaboração e troca entre equipes	Criar políticas de reuso (catálogo, revisão)
Melhorar documentação e testes	Combater “Not Invented Here” com cases de sucesso	Investir em infraestrutura inicial (repositórios, CI/CD)
Adotar versionamento e compatibilidade	Promover confiança em componentes validados	Medir e divulgar ganhos obtidos com reuso
Oferecer suporte técnico e guias	Estimular cultura de compartilhamento	Alinhar prazos para permitir reuso planejado

Dinâmica: Barreiras ao Reuso na Vida Real

- A tarefa: identificar quais barreiras ao reuso aparecem no cenário e propor soluções.

Cenários sugeridos

- **Startup em crescimento:** cada time cria sua própria API de autenticação, sem padrão.
- **Empresa tradicional:** equipe não confia em bibliotecas internas, prefere reescrever.
- **Universidade:** diferentes sistemas acadêmicos duplicam lógica de matrícula.
- **Big Tech:** possui repositório gigante de componentes, mas ninguém sabe como buscar o que precisa.

Cenários sugeridos

- **E-commerce:** Múltiplas equipes criam módulos de carrinho de compras distintos, sem integração.
- **Aplicativo mobile:** Dois times desenvolvem módulos de notificações push diferentes (um para iOS, outro para Android) sem reaproveitar código comum.
- **Sistema de saúde:** Cada hospital implementa seu próprio cadastro de pacientes, sem padronização de dados.
- **Banco:** Existem três módulos diferentes de cálculo de juros, desenvolvidos por áreas distintas, gerando inconsistências.

Cenários sugeridos

- **Governo:** Órgãos públicos mantêm sistemas separados para CPF e RG, em vez de um serviço centralizado de identificação.
- **Projeto acadêmico:** Cada aluno em disciplina de programação implementa seu próprio módulo de login, sem pensar em reuso ou padronização.
- **Empresa de logística:** Cada filial mantém seu próprio sistema de rastreamento de encomendas, sem compartilhar código ou APIs.
- **Startup de IA:** Equipes treinam modelos diferentes de recomendação, mas não compartilham datasets nem pipelines reutilizáveis.

Discussão

- “Qual barreira vocês acham mais difícil de superar na prática: técnica, cultural ou gerencial?”
- “Vocês já viram ou viveram a síndrome do Not Invented Here (Não Inventado Aqui) ?”
- “Quem deve liderar a institucionalização do reuso: desenvolvedores, arquitetos ou gestores?”

Resumo

- Reuso não é automático → exige processo, cultura e gestão.
- Barreiras existem, mas podem ser superadas com estratégia.
- Tecnologia ajuda, mas o fator humano (colaboração, confiança) é decisivo.
- Benefícios reais: menos retrabalho, mais qualidade, entregas rápidas.
- Lição final: Reuso é investimento no presente e no futuro do software.

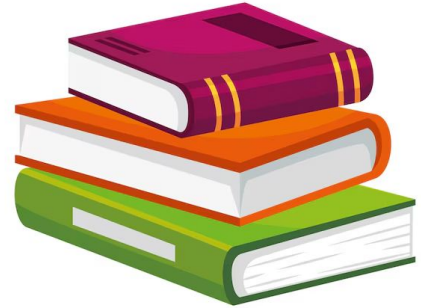
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

